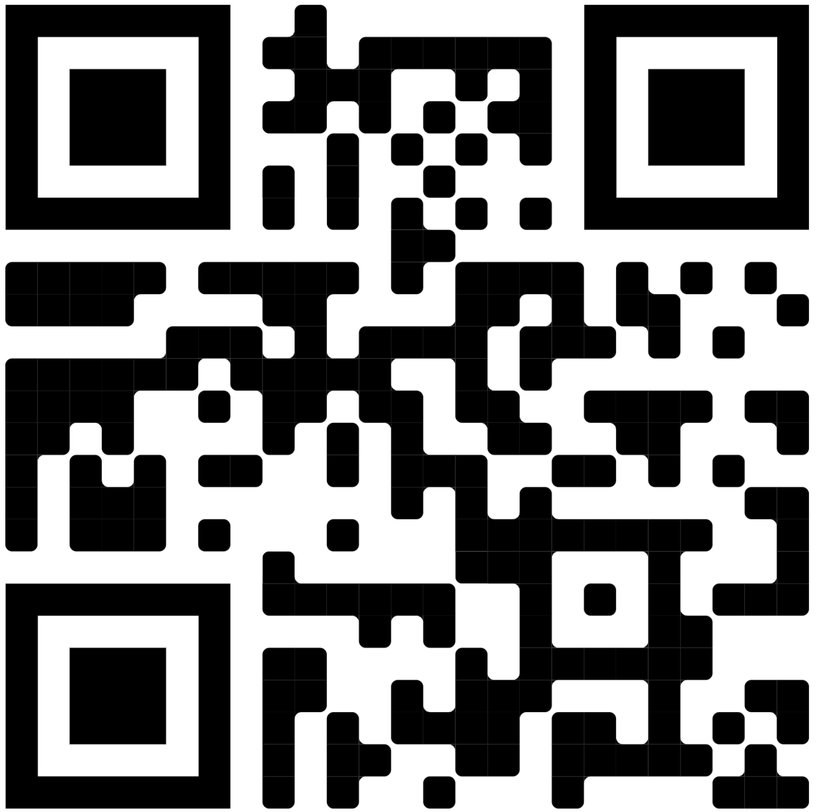


http://bit.ly/rs-tu-sam

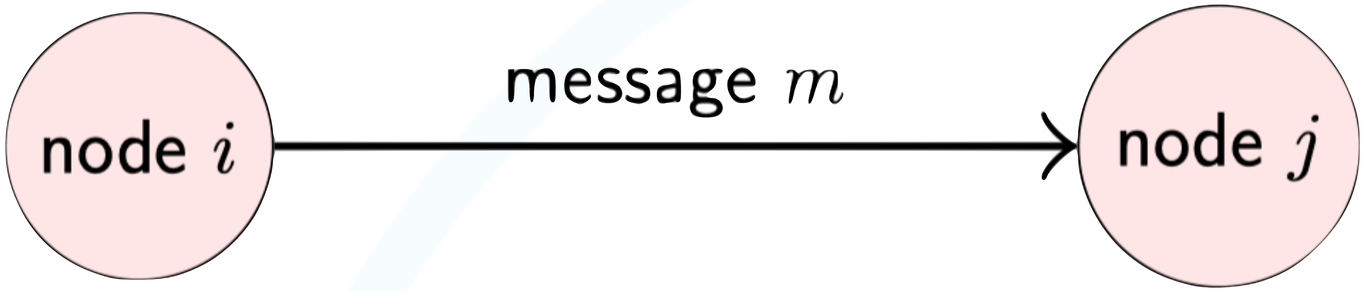
Distributed systems

Lectures: doc. dr. sc. Nikola Tanković

Communication

Networking

- Simple abstraction of communication



Author: Dr. Martin Kleppmann

- Meanwhile (reality): eduroam, home DSL, cellular, public Wi-Fi, satellites...

Latency & bandwidth

- **Latency:** time until (first part of) a message arrives
  - CPU to CPU via RAM: ~50 ns
  - Computer to computer (direct connection): ~1µs
  - Same building: ~1 ms
  - CPU to (spinning) disk: ~10 ms
  - Different continent: ~100 ms

Latency & bandwidth

- **Bandwidth:** max. data volume per unit of time
  - 3G cellular: ~ 1Mbit/s
  - 4G cellular: ~50Mbit/s
  - DSL: ~10Mbit/s
  - Hard drive in a van on 2 hour drive
    - 100TB drive → ~100Gbit/s

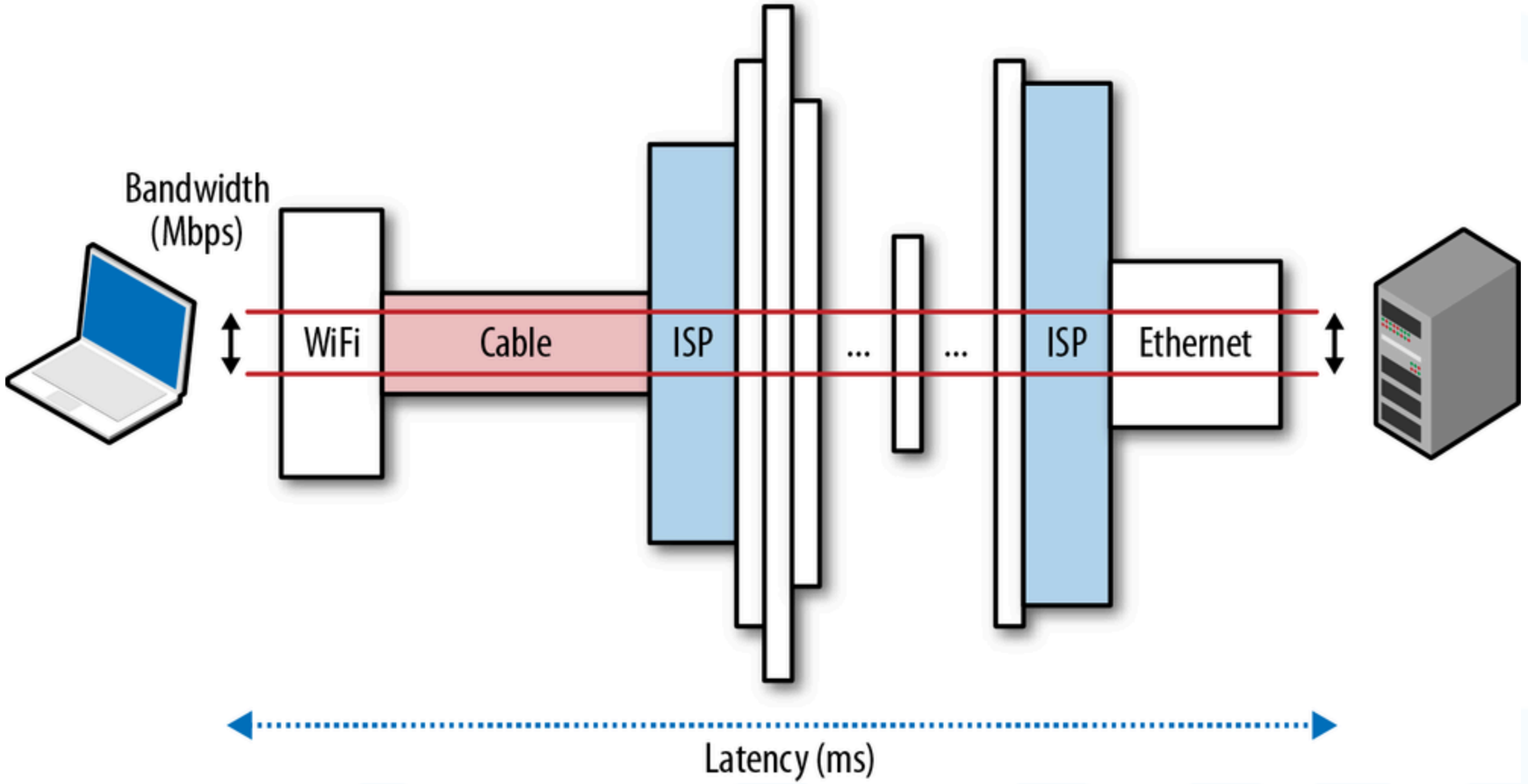
Latency & bandwidth

Latency

VS

Bandwidth

Latency & bandwidth



Theoretical minimum

Route	Distance	Time, light in vacuum	Time, light in fiber	Round-trip time (RTT) in fiber
New York to San Francisco	4,148 km	14 ms	**21 ms**	42 ms
New York to London	5,585 km	19 ms	**28 ms**	56 ms
New York to Sydney	15,993 km	53 ms	**80 ms**	160 ms
Equatorial circumference	40,075 km	133.7 ms	**200 ms**	200 ms

Humans perceive 100-200ms as slight "lag", > 300 "sluggish"

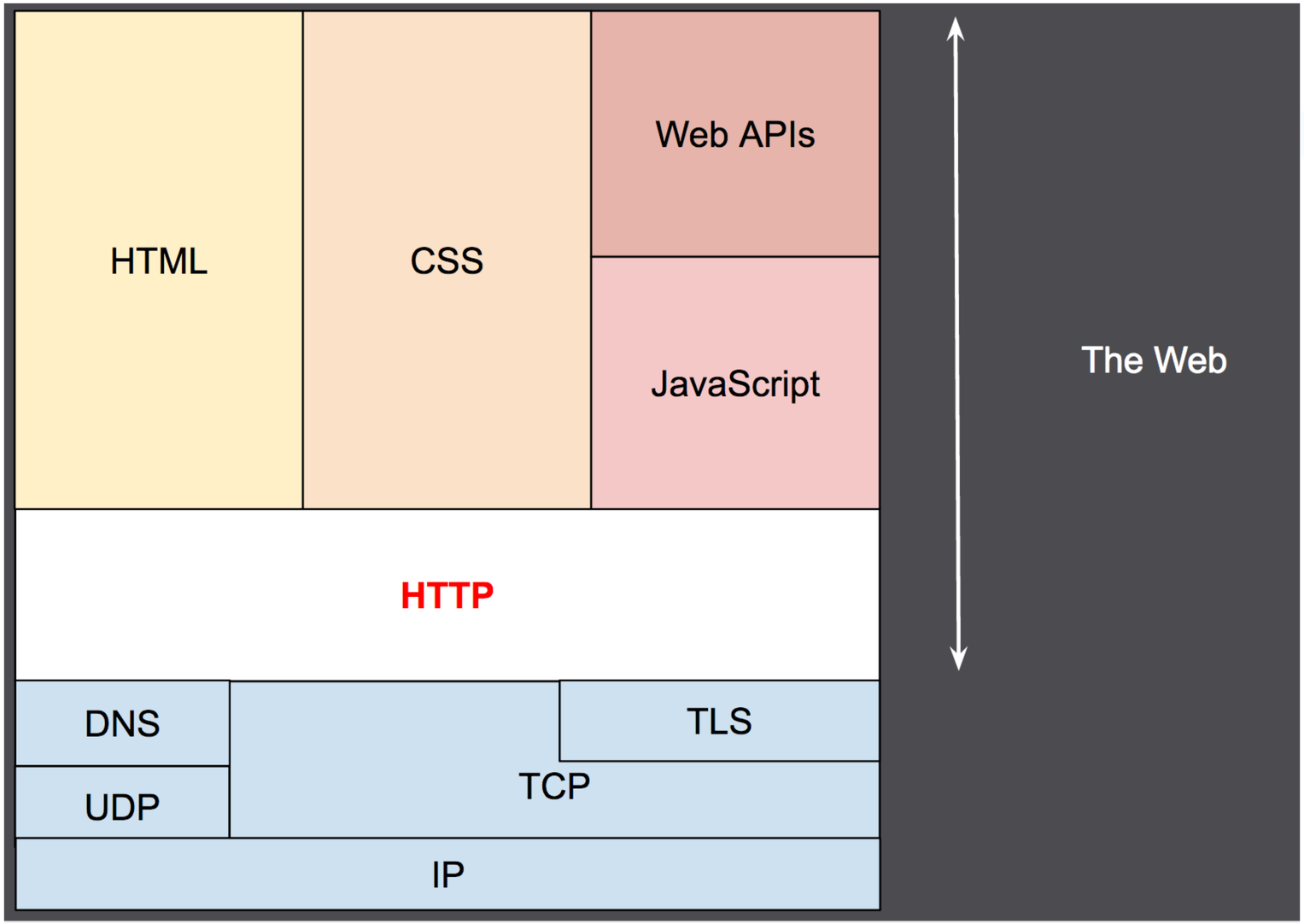
traceroute

```
traceroute to google.com (142.250.180.238), 64 hops max, 52 byte packets
 1  192.168.1.1 (192.168.1.1)  1.138 ms  0.562 ms  0.273 ms
 2  10.42.0.1 (10.42.0.1)  2.202 ms  2.288 ms  1.988 ms
 3  45.142.10.1 (45.142.10.1)  2.093 ms  6.959 ms  2.269 ms
 4  dh70-161.xnet.hr (83.139.70.161)  13.086 ms  10.774 ms  7.745 ms
 5  * 195.3.114.229 (195.3.114.229)  16.006 ms  17.697 ms
 6  * * *
 7  * * *
 8  lg59-9071.as8447.a1.net (80.120.167.46)  22.983 ms  21.049 ms  19.706 ms
 9  * * *
10  216.239.35.250 (216.239.35.250)  20.686 ms
    142.251.228.26 (142.251.228.26)  25.162 ms
    142.251.228.28 (142.251.228.28)  23.280 ms
11  bud02s34-in-f14.1e100.net (142.250.180.238)  20.741 ms
    142.251.65.217 (142.251.65.217)  31.523 ms
    142.251.65.219 (142.251.65.219)  20.150 ms
```

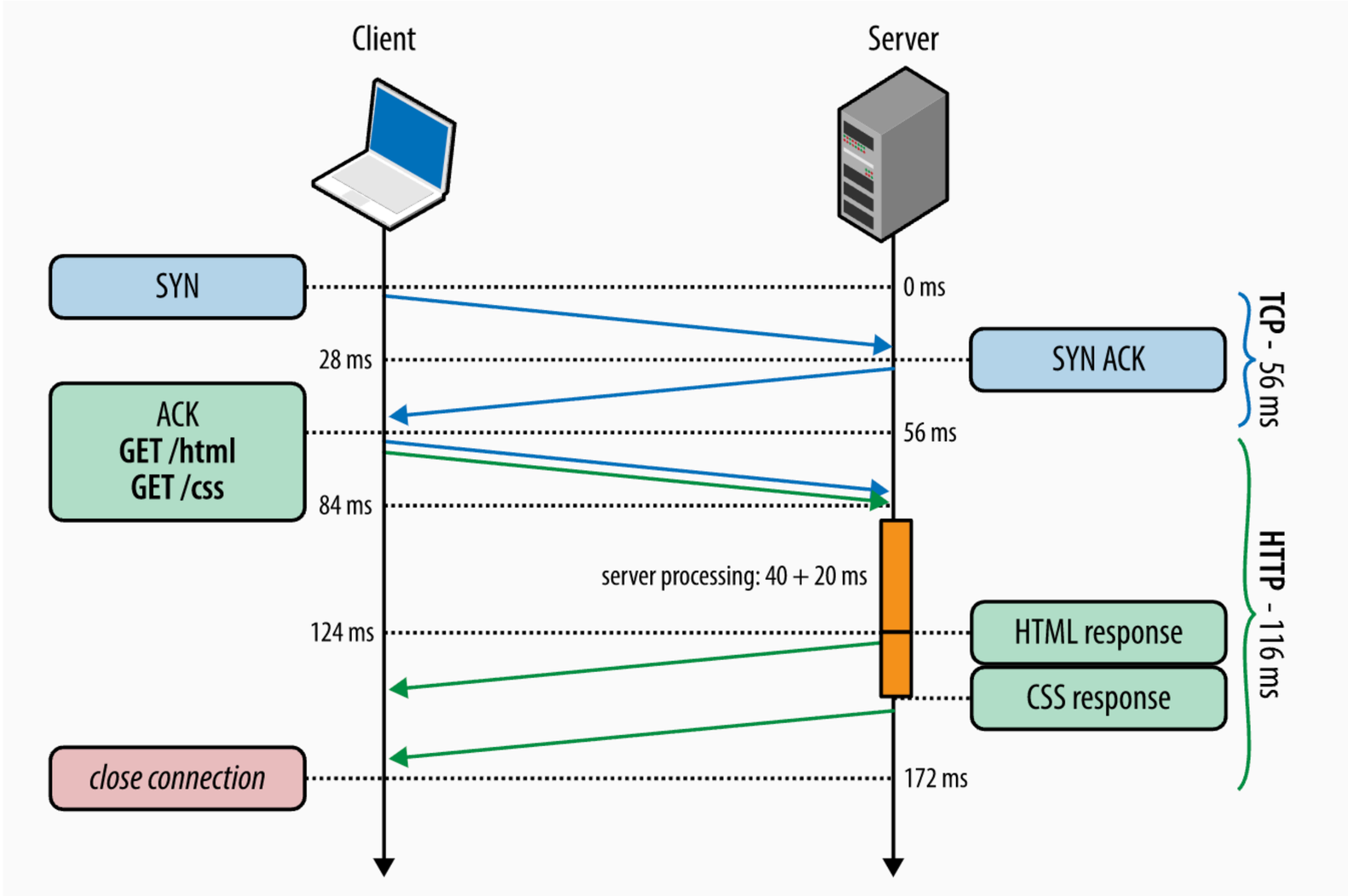
High-bandwidth high-latency example

A familiar message protocol

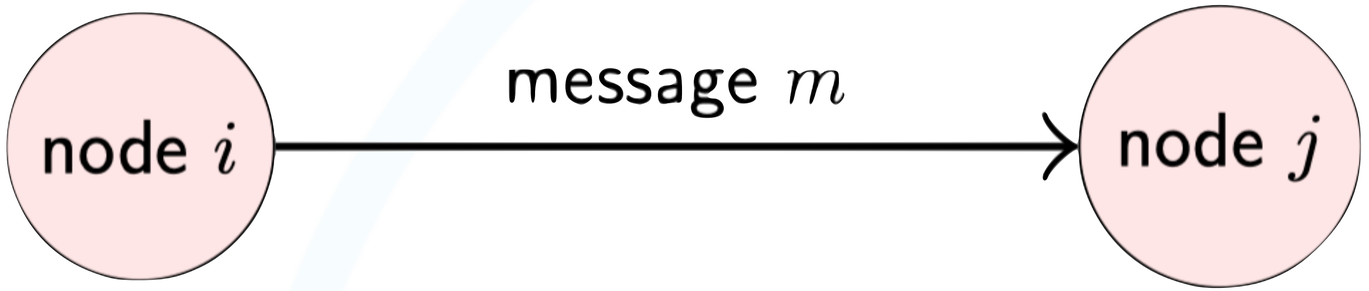
- Can you remember one?
- Where have we seen requests and replies?


HTML
CSS
Web APIs
JavaScript
The Web
HTTP
DNS
TLS
UDP
TCP
IP

Example

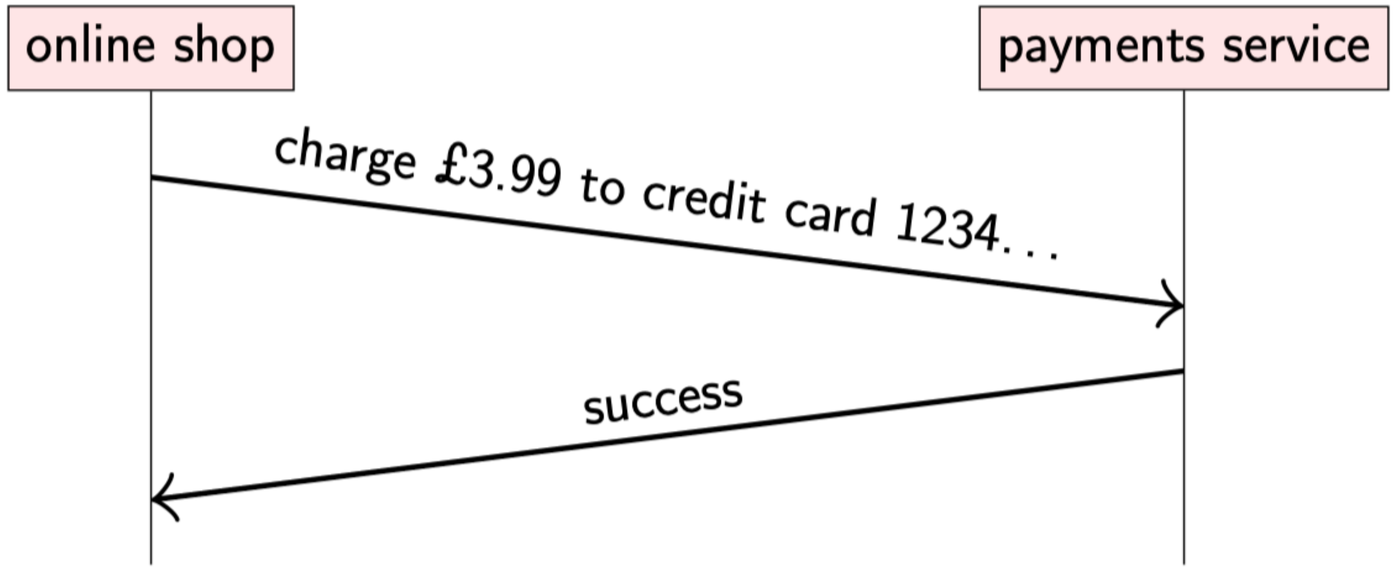

Client
Server
SYN
0 ms
SYN ACK
28 ms
ACK
GET /html
GET /css
56 ms
84 ms
server processing: 40 + 20 ms
HTML response
124 ms
CSS response
close connection
172 ms
TCP - 56 ms
HTTP - 116 ms

Much simpler



Author: Dr. Martin Kleppmann

RPC - remote procedure call



Author: Dr. Martin Kleppmann

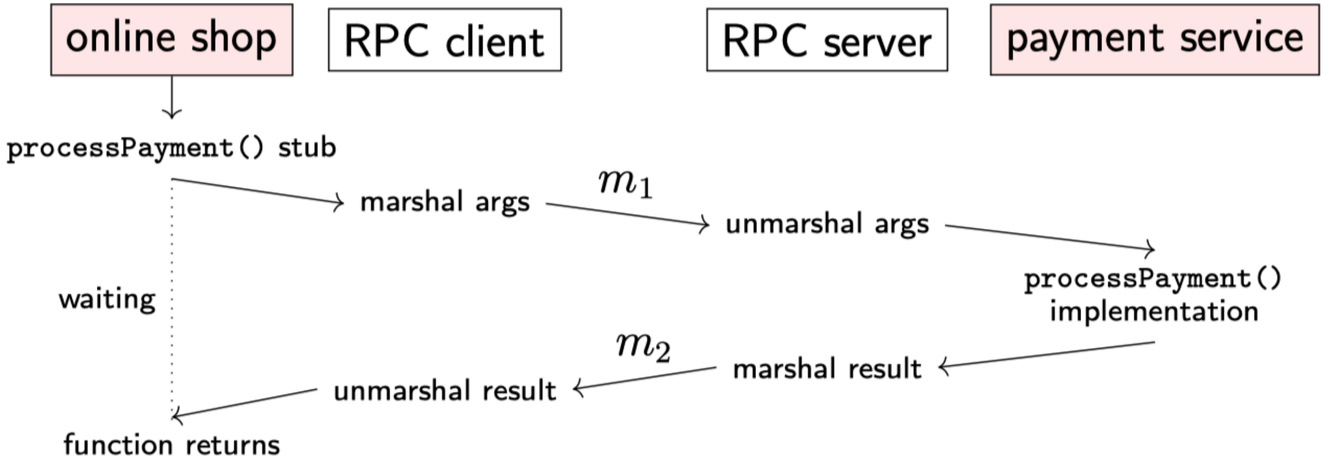
RPC - remote procedure call

```
let card = {
    "number": "4312-5678-1234-1234",
    "expiryDate": "10/2024",
    "cvc": "123"
}

let response = await Service.processPayment(card, 3.99, "GBP");

if (response.result == "success") {
    showSuccessScreen();
}
```

Many steps in-between



$m_1 =$

```
{
  "request": "processPayment",
  "card": {
    "number": "1234567887654321",
    "expiryDate": "10/2024",
    "CVC": "123"
  },
  "amount": 3.99,
  "currency": "GBP"
}
```

$m_2 =$

```
{
  "result": "success",
  "id": "XP61hHw2Rvo"
}
```

Author: Dr. Martin Kleppmann

RPC

- Remember **location** transparency?
- What can go wrong?
  - Lost?
  - Delayed?
  - Server crash?
  - Safe to retry?
- Remember **idempotency**?

RPC examples

- SOAP/XML-RPC: RPC using XML and HTTP (1998)
- Thrift (Facebook, 2007)
- **gRPC** (Google, 2015)
- **REST** (often with JSON)
- Ajax in web browsers (**fetch**/**axios** and others…)

Enterprise RPC

- Usually has a strict schema for each P in RPC
- **Interface Definition Language (IDL)**
  - A language independent API specification

gRPC example

```
message PaymentRequest {
  message Card {
    required string cardNumber  = 1;
    optional int32  expiryMonth = 2;
    optional int32  expiryYear  = 3;
    optional int32  CVC         = 4;
  }
  enum Currency { GBP = 1; USD = 2; }

  required Card     card     = 1;
  required int64    amount   = 2;
  required Currency currency = 3;
}

message PaymentStatus {
  required bool   success      = 1;
  optional string errorMessage = 2;
}

service PaymentService {
  rpc ProcessPayment(PaymentRequest) returns (PaymentStatus) {}
}
```

Demo

aiohttp

Distributed systems

Lectures: doc. dr. sc. Nikola Tanković

Models of distributed systems

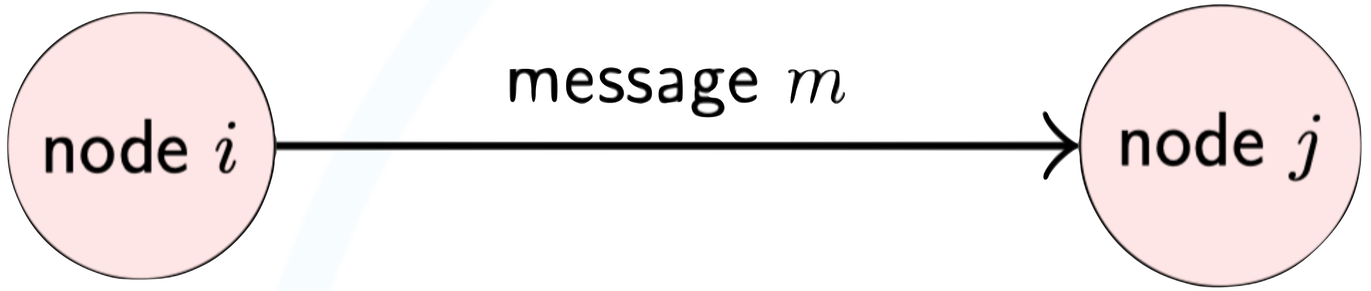
System model

- A **system model** captures our **assumptions** about how **nodes** and the **network** behave
- An abstract description of their **properties**, which can be implemented by **various technologies** in practice

- Let see some motivating examples ...

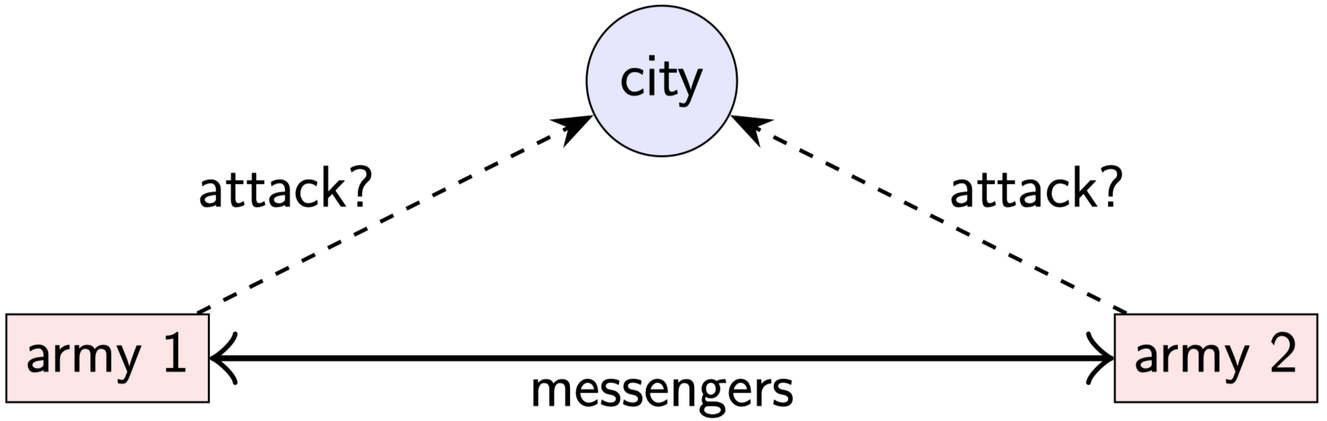
Remember...

- Still using simple communication model



Author: Dr. Martin Kleppmann

Example 1: The two generals problem



Author: Dr. Martin Kleppmann

Outcomes

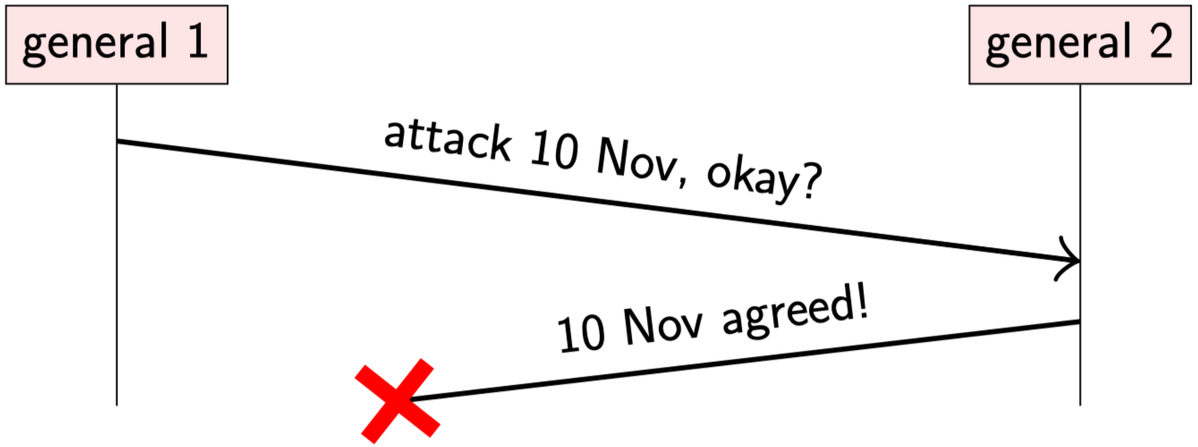
Author: Dr. Martin Kleppmann

army 1	army 2	outcome
does not attack	does not attack	nothing happens
attacks	does not attack	army 1 defeated
does not attack	attacks	army 2 defeated
attacks	attacks	city captured

Desired: army 1 attacks *if and only if* army 2 attacks

Note: Coordination is the only way for the generals to succeed.

… the challenge

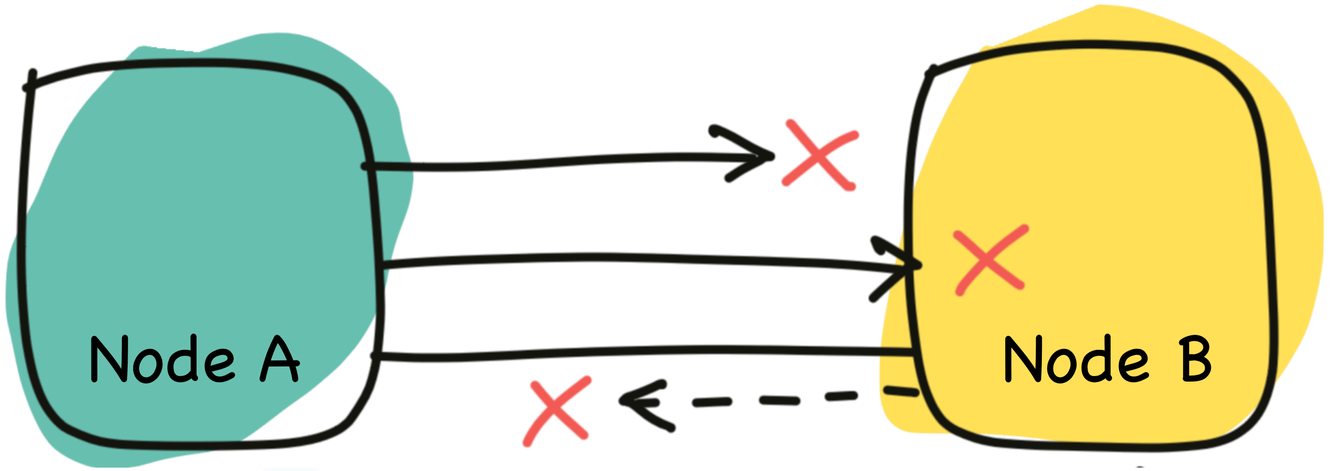


Author: Dr. Martin Kleppmann

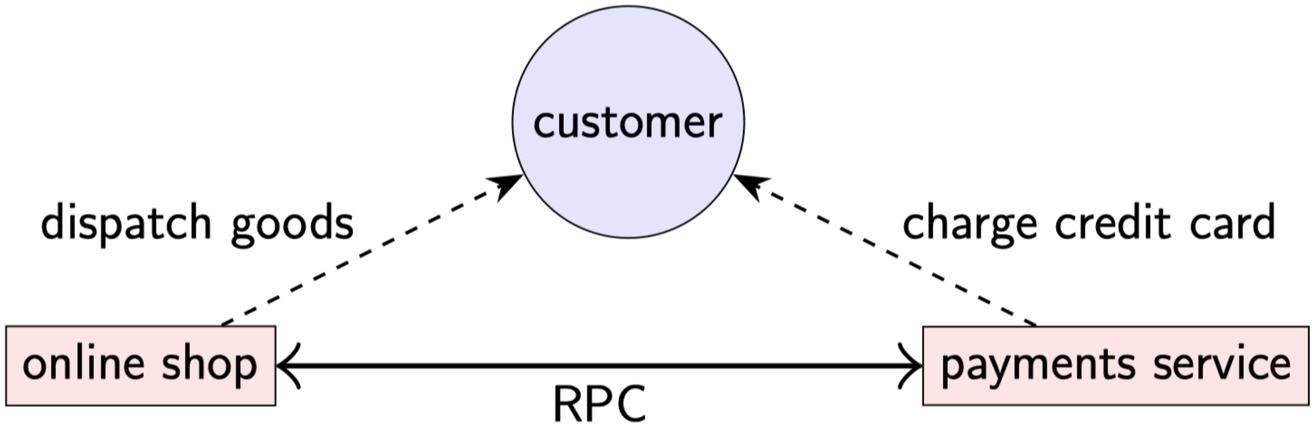
"No common knowledge"

- The only way to know something is by communicating
- Whenever there is a possibility (no matter how small) that a message will get lost, there can be no 100% confidence in the outcome
- **No node** can know the state of **another node** with **certainty**

Impossible to differentiate



Web-shop analogy



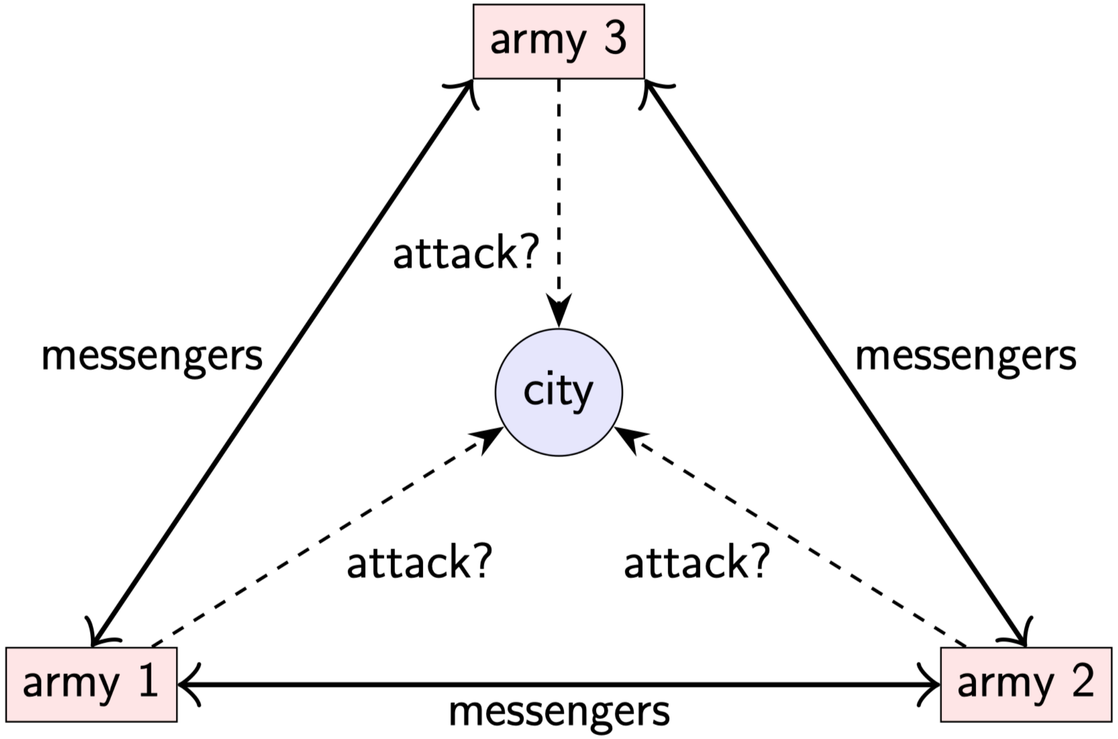
online shop	payments service	outcome
does not dispatch	does not charge	nothing happens
dispatches	does not charge	shop loses money
does not dispatch	charges	customer complaint
dispatches	charges	everyone happy

Desired: online shop dispatches *if and only if* payment made

Question: Why can this be resolved?

Author: Dr. Martin Kleppmann

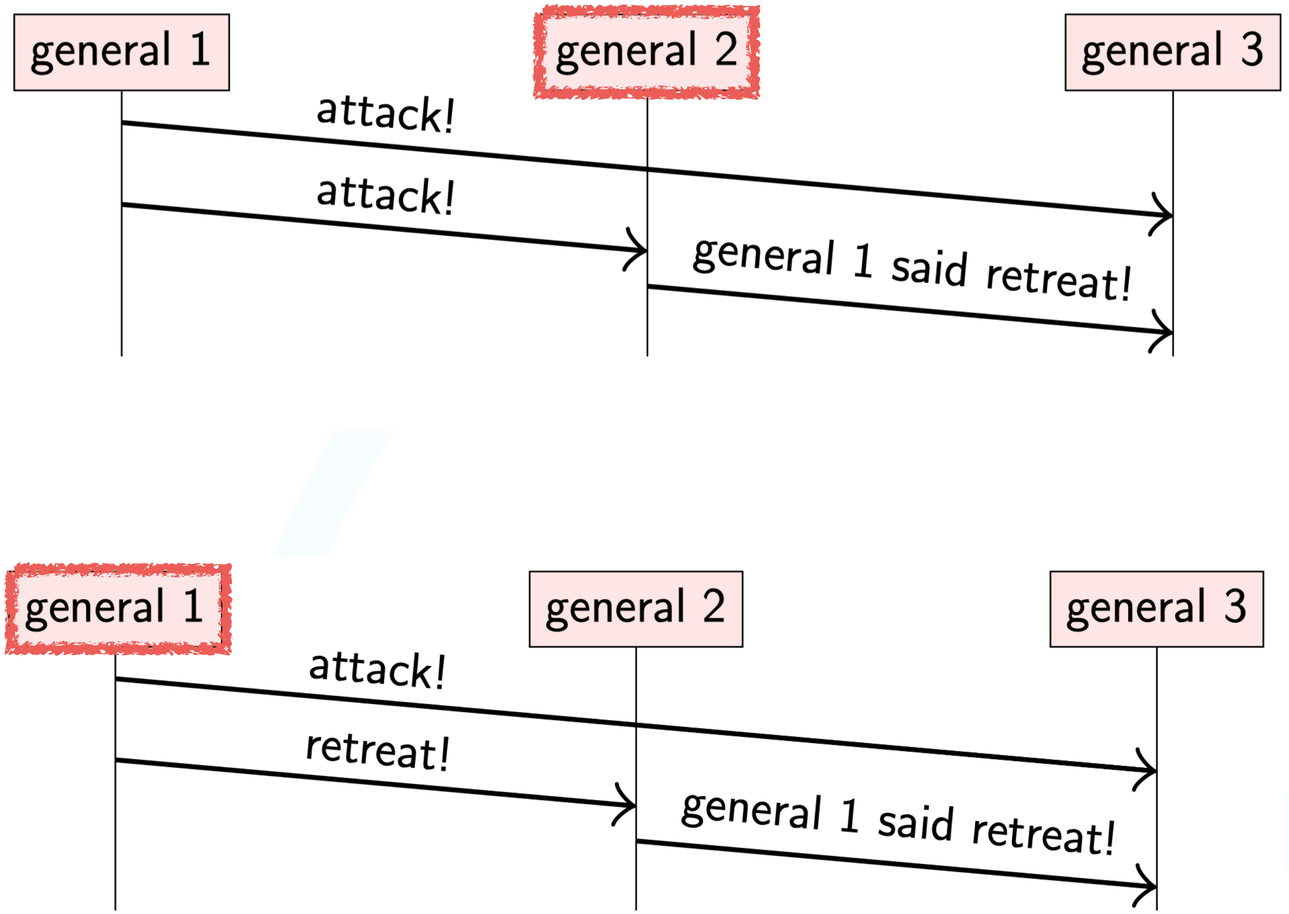
Example 2: The Byzantine generals problem



Author: Dr. Martin Kleppmann

Problem: some of the generals can be traitors

Generals can cheat



Lamport et al. 1982

Figure: Dr. Martin Kleppmann

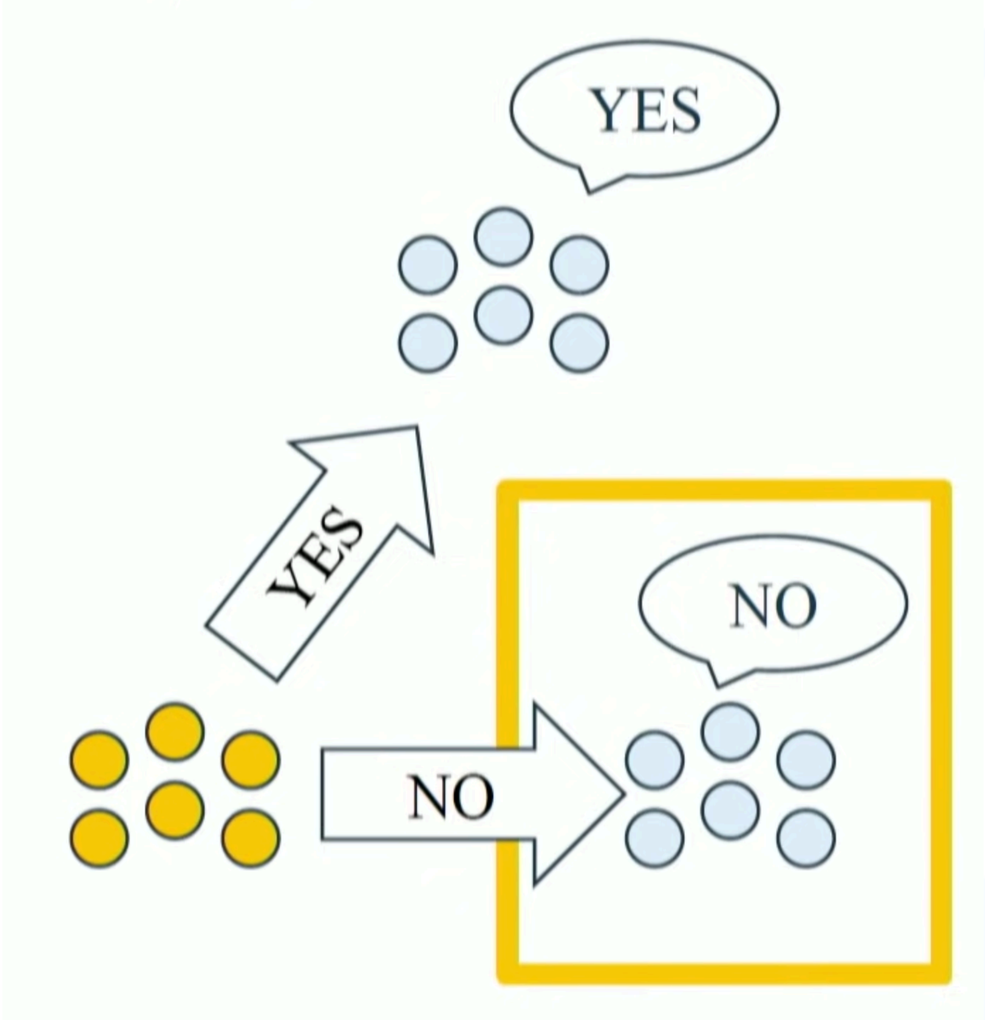
Problem summary

- Each general is either **honest** or **malicious**

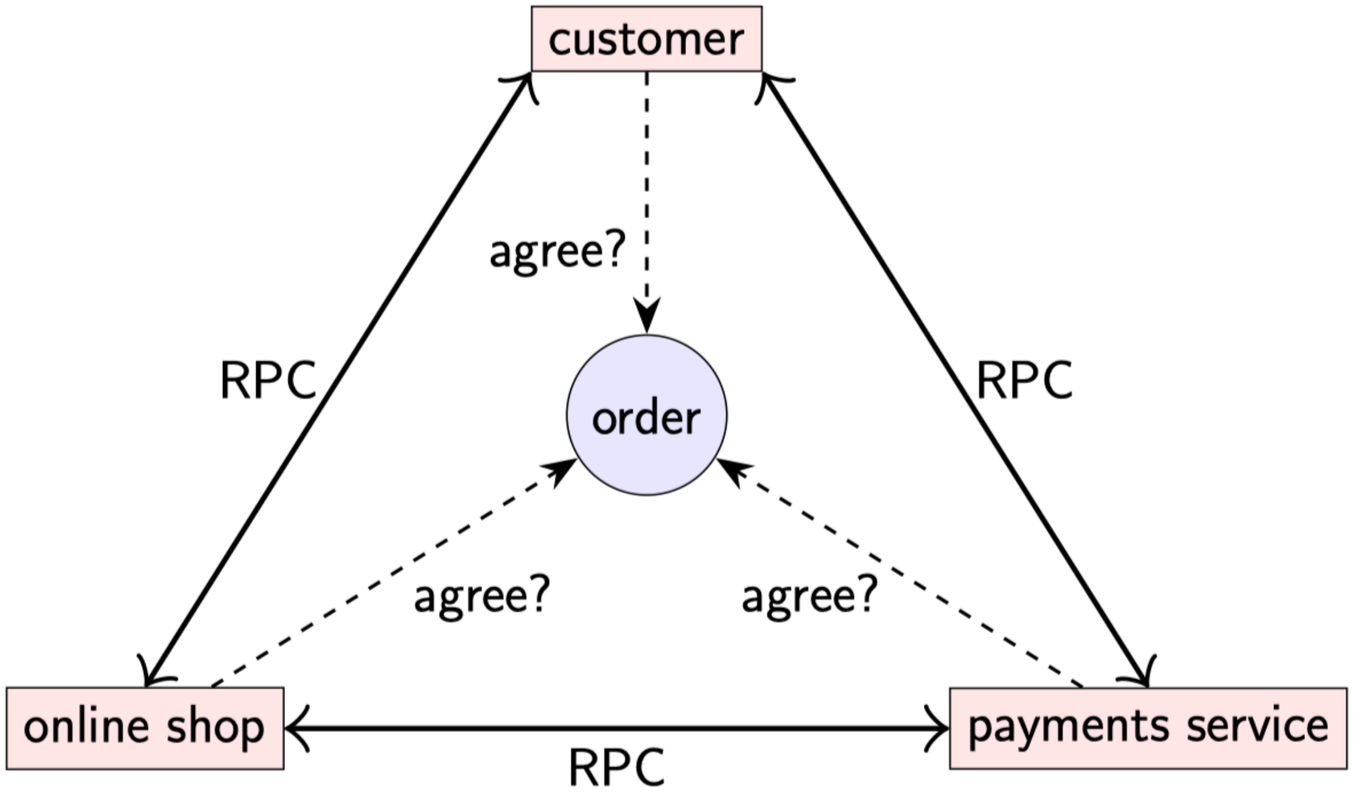
- Generals **do not know** which ones are malicious
- **Theorem:** If some n generals are malicious, we need $3n + 1$ generals in total

  → requirement: $< \frac{1}{3}$ of generals can be malicious

Proof



How does this translate?



Who can trust whom?

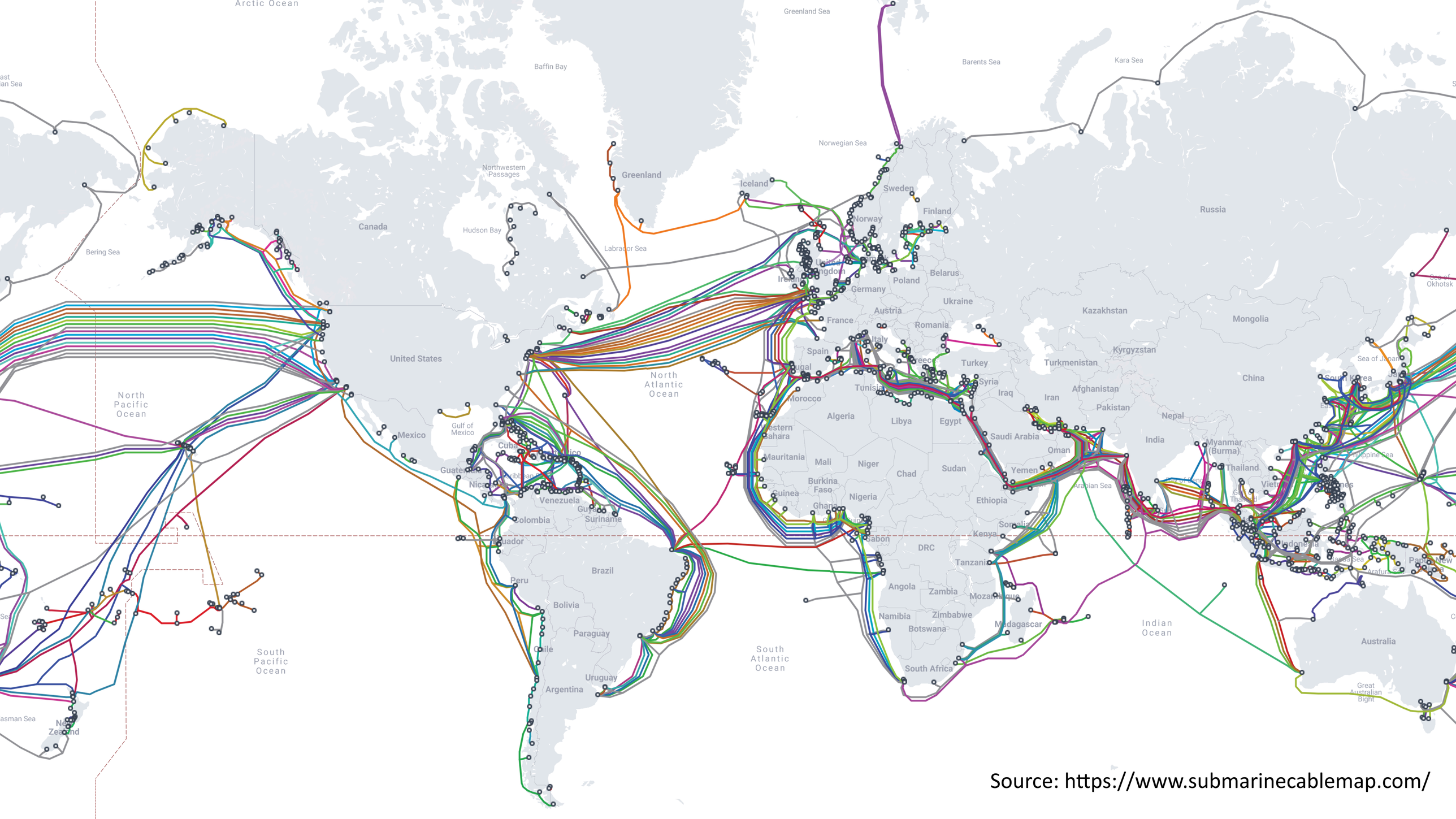
Figure: Dr. Martin Kleppmann

A more formal approach

System model

- Two generals problem → a model of networks (messages get lost)
- Byzantine generals problem → a model of node behavior
  - A node can be faulty (e.g. down or have a ***bug***)
  - The node operator can be malicious
- In **practice**: both situations: both network and the nodes are unreliable

Networks are unreliable



Source: https://www.submarinecablemap.com/

M1

Network model

- Most algorithms assume one of three possibilities:
  - **Perfect** link - message, when sent, will get through (ordering is not preserved) 😰
  - **Fail-loss** links - some messages will get lost, duplicated or reordered (retries help) but eventually get there
    - Solutions: **idempotency**, **delivery guarantees**
  - **Arbitrary** links (with an active adversary)
    - Eavesdropping, modifying, replaying, ...
    - Solutions: **SSL/TLS**

M2

Node model

- Each **node executes** an **algorithm**, and may:
  - **Fail-Stop** (crash-stop)
    - Upon failure, it stops executing and potentially loses memory for good (think drones?)
  - **Fail-Recovery** (crash-recovery)
    - Nodes can resume operation (think servers?)
  - **Byzantine** (fail-arbitrary)
    - Deviation from the algorithm (***on purpose on not***)

- Hard to change the algorithms between these assumptions

M3

Timing model

- Applicable to both network and nodes:
  - **Synchronous**
    - Known **upper** bound for network latency
    - Nodes operate at a **known** speed
  - **Asynchronous**
    - Arbitrary delays and/or pauses in execution
    - No timing guarantees
  - **Partially synchronous**
    - Sometimes synchronous, but can go **async** for an **unknown** amount of time

Reality on timings

- A **synchronous model** is a dream, which we have *most of the time*
- Algorithms designed with that dream in mind will fail
  - Latency and speed can vary **wildly**
- The **asynchronous model** is another extreme
  - Algorithms are very robust, but not all problems can be solved

Usual violations of synchronicity

- **Networks**
  - Message loss (wireless networks, routers fail)
  - Congestion (queueing, increased latency)
  - Network reconfigurations
- **Nodes**
  - OS scheduling issues
  - "Garbage collection"
  - Page faults, memory swapping

Distributed model summary

- When designing an algorithm, pick your assumptions
  - **Network:** reliable, fail-loss, arbitrary
  - **Nodes:** crash-stop, crash-recovery, or Byzantine
  - **Timing:** sync, async, partially sync

The story of five nines

High-availability

- The holy grail of any system and the most **common reason** to build a distributed system
- Amazon wants to be up 24/7
  - Downtime means losing money (about $300K/min)
- **Availability**: fraction of the time that the system is up
  - "Two nines" 99% up ~ 4 days/year
  - "Five nines" 99.999% up ~ 5 minutes/year

Service-level agreements

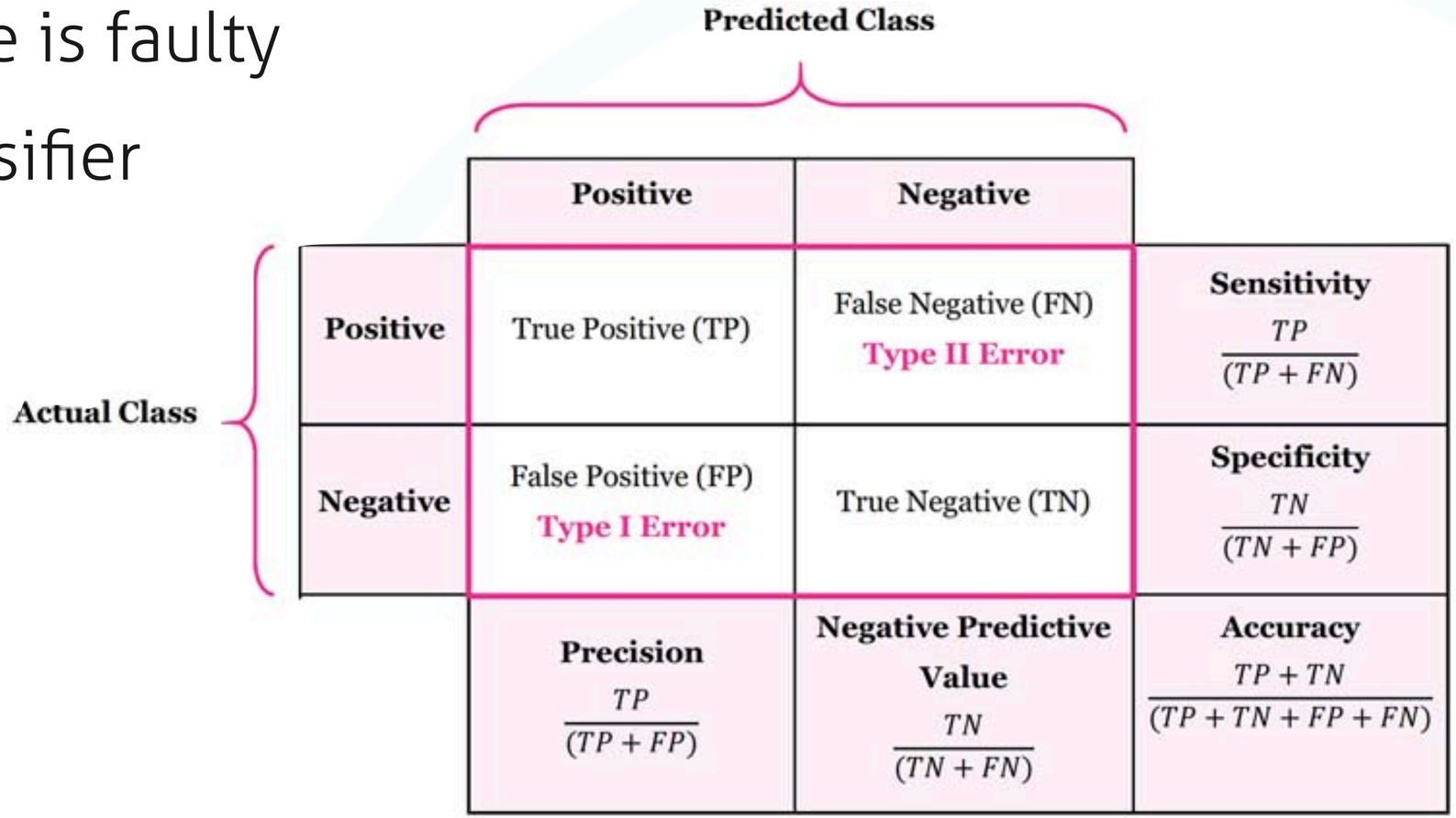
- **SLO** - service-level objectives
  - A well-defined metric that is combining several attributes (usual latency with availability)
    - e.g. 99.9% of all requests in a day will be served under 200ms
- **SLA** - service-level agreement
  - A contract consisting of 1+ SLO's
  - Penalties involved

Fault tolerance

- **Failure:** the system is not working
  - I can not order on Amazon
- **Fault:** a part of the system is not working
  - Node or network crash
  - I can use the system with some level of degradation
- **Fault tolerance**: the ability of the system to keep working despite the faults or up-to-some number of faults
- **SPOF - single point of failure** - a network link or a node whose **fault** lead to **failure**

Detecting failures

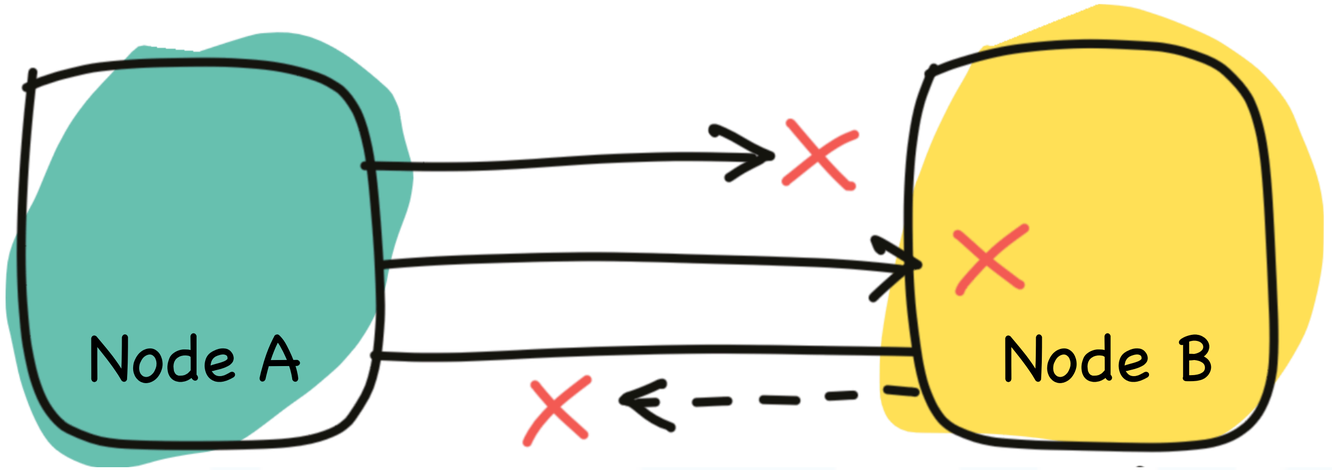
- **Failure detector**: algorithm that detects another node is faulty
- Classifier



Actual Class \ Predicted Class	Positive	Negative	
Positive	True Positive (TP)	False Negative (FN) **Type II Error**	**Sensitivity** $\frac{TP}{(TP+FN)}$
Negative	False Positive (FP) **Type I Error**	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
	Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

Usual approach & challenge

- Send a message, wait for X seconds, mark as failed/ok
- **Problem:** no difference between node crash/ fault, message delay, or lost message



Self-assesment

What is RPC?

What is the difference between latency and bandwidth?

Describe the two general problem.

Describe the Byzantine generals' problem.

What can go wrong with the distributed system?

What is the difference between SLA and SLO?

What is SPOF?